REINFORCEMENT LEARNING

Guía Completa de Estudio

Desde los fundamentos hasta algoritmos avanzados

1. INTRODUCCIÓN AL REINFORCEMENT LEARNING

1.1 ¿Qué es el Reinforcement Learning?

El Reinforcement Learning (RL) o Aprendizaje por Refuerzo es un paradigma del Machine Learning donde un agente aprende a tomar decisiones mediante la interacción con un entorno. A diferencia del aprendizaje supervisado, no hay un supervisor externo que proporcione etiquetas correctas; en su lugar, el agente recibe señales de recompensa que indican qué tan buenas fueron sus acciones.

1.1.1 RL en el contexto del Machine Learning

El Machine Learning se divide en tres paradigmas principales:

- **Aprendizaje Supervisado:** Utiliza datasets etiquetados por un supervisor externo. Se usa frecuentemente para identificar categorías y hacer predicciones.
- **Aprendizaje No Supervisado:** Trabaja con datos sin etiquetar. Su objetivo es encontrar patrones ocultos en los datos.
- **Reinforcement Learning:** Datos sin etiquetar con señal de recompensa. El objetivo es maximizar esta señal. El agente aprende haciendo (learn by doing).

1.1.2 Aplicaciones del RL

El Reinforcement Learning tiene aplicaciones en múltiples dominios:

- Videojuegos: Atari (DQN de DeepMind), AlphaGo, AlphaZero
- Robótica: Manipulación de objetos, locomoción (Boston Dynamics)
- Control industrial: Control de reactores de fusión (tokamak)
- Vehículos autónomos
- Large Language Models (LLMs): RLHF para alinear modelos

1.2 Cómo funciona el RL: El ciclo Agent-Environment

El funcionamiento del RL se basa en un ciclo de interacción entre dos entidades principales:

1.2.1 Componentes fundamentales

- **Agente (Agent):** El sistema que toma decisiones y aprende. Contiene la política (policy) que define su comportamiento.
- **Entorno (Environment):** El mundo con el que interactúa el agente. Proporciona estados y recompensas.
- **Estado (State) St:** Representación de la situación actual del entorno.
- **Acción (Action) At:** Decisión tomada por el agente basándose en el estado actual.
- **Recompensa (Reward) Rt:** Señal numérica que indica qué tan buena fue la acción.

1.2.2 El ciclo de interacción

El ciclo funciona de la siguiente manera:

1. El agente observa el estado actual St del entorno
2. Basándose en su política, el agente selecciona una acción At
3. El entorno transiciona al nuevo estado St+1
4. El entorno proporciona una recompensa Rt+1
5. El agente actualiza su política basándose en la experiencia
6. El ciclo se repite (Trial and Error)

Episodio (Trajectory): Una secuencia completa de interacciones: S_0 A_0 R_1 S_1 A_1 R_2 S_2 A_2 R_3 S_3 ... ST-1 AT-1 RT ST

2. MARKOV DECISION PROCESSES (MDP)

2.1 Introducción a los MDP

Los Markov Decision Processes son el marco matemático formal para modelar problemas de toma de decisiones secuenciales bajo incertidumbre.

2.1.1 El problema de la incertidumbre

Consideremos un robot navegando en un entorno con un tesoro (recompensa positiva) y fuego (recompensa negativa). Si pudiéramos predecir perfectamente los resultados de nuestras acciones, podríamos usar algoritmos como A* para encontrar el camino óptimo.

Sin embargo, en el mundo real siempre existe incertidumbre:

- Incertidumbre en la localización: ¿Podríamos estar más cerca de una zona peligrosa de lo que creemos?
- Incertidumbre en las acciones: El robot podría ejecutar la acción de forma imperfecta

2.1.2 Modelo de transición estocástico

Para modelar esta incertidumbre, definimos una distribución de probabilidad sobre los resultados de cada acción. Por ejemplo, si el agente intenta moverse hacia arriba:

- 80% de probabilidad de moverse hacia arriba (resultado deseado)
- 10% de probabilidad de moverse hacia la izquierda
- 10% de probabilidad de moverse hacia la derecha

Esta incertidumbre hace que algunas rutas sean más 'seguras' que otras, aunque sean más largas.

2.2 Definiciones formales del MDP

2.2.1 Modelo de transición T(s, a, s')

La probabilidad de alcanzar el estado s' desde el estado s ejecutando la acción a se denomina modelo de transición:

$$T(s, a, s') = P(S_{t+1} = s' \mid S_t = s, A_t = a)$$

2.2.2 Propiedad de Markov

La propiedad de Markov establece que las probabilidades de transición de s a s' dependen únicamente del estado actual s, no del historial de estados anteriores:

$$Pr(S_{t+1} \mid S_t, A_t) = Pr(S_{t+1} \mid S_{0:t}, A_{0:t})$$

Esto significa que el estado actual contiene toda la información relevante para predecir el futuro. El pasado no aporta información adicional una vez conocemos el presente.

2.2.3 Función de recompensa R(s)

En cada transición a un estado s', el agente recibe una recompensa R(s). Esta recompensa:

- Puede ser positiva o negativa (pero acotada)
- Puede generalizarse como R(s, a, s') para depender de la acción y transición

Ejemplo: En un grid world, las celdas normales podrían tener R = -0.04 (pequeño costo por paso), el tesoro R = +1, y el fuego R = -1.

2.2.4 Definición completa del MDP

Un MDP para un problema de decisión secuencial en un entorno completamente observable y estocástico se define por:

- **S:** Conjunto de estados (ej: posiciones del robot)
- **A:** Conjunto de acciones
- **s_0:** Estado inicial
- **T(s, a, s'):** Modelo de transición
- **R(s):** Función de recompensa

Solución: La solución a un MDP se llama política (π).

2.3 Funciones de Valor

2.3.1 Función de valor de estado V(s)

El valor (utilidad) de un estado V(s) cuantifica el beneficio de estar en ese estado para la tarea global. Específicamente, mide cuánta recompensa esperamos obtener en el futuro partiendo de ese estado.

V(s) evalúa todos los posibles futuros partiendo de s

2.3.2 Ecuación de Bellman

La ecuación de Bellman expresa el valor de un estado en términos de las recompensas inmediatas y los valores de los estados sucesores:

$$V(s) = R(s) + \max_a Q(s, a)$$

donde:

$$Q(s, a) = \Sigma s' T(s, a, s') \cdot V(s')$$

Esta ecuación recursiva es fundamental para todos los algoritmos de RL basados en valor.

2.4 Value Iteration

2.4.1 Descripción del algoritmo

Value Iteration es un algoritmo iterativo de programación dinámica para encontrar la función de valor óptima y la política óptima de un MDP.

2.4.2 Pasos del algoritmo

7. Inicializar V(s) para todos los estados s con valores arbitrarios o ceros
8. Repetir hasta que el cambio en V(s) sea pequeño o se alcance el máximo de iteraciones:
    - Para cada estado s, calcular el valor para cada acción a usando la ecuación de Bellman
    - Actualizar: V(s) = R(s) + max(Q(s, a)) para todas las acciones a

2.4.3 Convergencia y política óptima

- Value Iteration garantiza convergencia a la función de valor óptima
- La política óptima se obtiene seleccionando la acción con el valor más alto para cada estado

2.4.4 Ventajas y limitaciones

Ventajas:

- Simple de implementar y aplicable a MDPs grandes
- Proporciona una política óptima que maximiza la recompensa acumulada esperada

Limitaciones:

- Puede requerir muchas iteraciones para converger en MDPs complejos
- La complejidad temporal aumenta con el número de estados y acciones
- CRÍTICO: Requiere conocer el modelo de transición, que normalmente es desconocido

3. MODELADO DE PROBLEMAS DE RL

3.1 Jerarquía de procesos de Markov

Los problemas de RL se modelan como Markov Decision Processes, que son una extensión de procesos más simples:

Proceso	Componentes	Añade	Fórmula
Markov Chain	Estados S	-	-
Markov Process	S + Transiciones	p(s'|s)	Σs' p(s'|s) = 1
Markov Reward Process	+ Recompensas	r(s, s')	-
MDP	+ Acciones	A, p(s'|s,a), r(s,a,s')	Σs' p(s'|s,a) = 1

3.2 Objetivo del RL: Maximizar el Retorno

El objetivo fundamental del RL es maximizar la recompensa acumulada (utilidad o retorno).

3.2.1 Retorno acumulado

En cada timestep t, el agente recibe una recompensa Rt. El retorno acumulado es:

$$Gt = Rt+1 + Rt+2 + Rt+3 + ... + RT$$

- **Tarea episódica:** Si T es finito (el episodio termina)
- **Tarea continua:** Si T tiende a infinito

Expected Return: El promedio de Gt sobre múltiples episodios.

3.2.2 Descuento (Discounting)

En tareas continuas, el retorno esperado podría ser infinito. Para evitar esto, se usa el descuento:

$$Gt = Rt+1 + \gamma Rt+2 + \gamma^2 Rt+3 + ... = \Sigma k=0\rightarrow\infty\ \gamma^k Rt+k+1$$

O de forma incremental:

$$Gt = Rt+1 + \gamma Gt+1$$

El parámetro gamma (γ) está entre 0 y 1:

- **γ = 0:** Agente 'miope', solo maximiza recompensas inmediatas
- **γ = 1:** Considera todas las recompensas igualmente (solo para tareas episódicas)

3.3 Políticas

Obtener una política es el objetivo principal del Reinforcement Learning.

3.3.1 Definición

Una política π es un mapeo de estados a acciones: π: S → A

3.3.2 Política determinista

$$\pi(s) = a$$

Para cada estado, devuelve una única acción. Puede representarse como una tabla (estados discretos) o una función (estados continuos).

3.3.3 Política estocástica

$$\pi(a|s) = Pr(At = a \mid St = s)$$

Para cada estado, devuelve una distribución de probabilidad sobre las acciones. Las acciones se muestrean aleatoriamente según su probabilidad.

3.3.4 Política óptima π*

En un entorno estocástico, la misma política puede producir diferentes historias. La calidad de una política se mide por el retorno esperado de todas las posibles historias del entorno.

Política óptima (π*): Es aquella que es mejor o igual que todas las demás políticas.

La política óptima puede cambiar dependiendo de la función de recompensa. Por ejemplo, si el costo por paso es muy alto, el agente preferirá rutas más cortas aunque sean más arriesgadas.

4. ENTORNOS: GYMNASIUM

4.1 Introducción a Gymnasium

OpenAI definió un conjunto de entornos de prueba para RL llamado OpenAI Gym. Gymnasium (de la Farama Foundation) es su sucesor y proporciona una API consistente para interactuar con diferentes problemas de forma transparente.

4.1.1 Tipos de entornos disponibles

- **Algorithms:** Entornos basados en texto para entrenar computaciones
- **Atari:** Basados en juegos de Atari como Pong
- **Box2D:** Tareas de control 2D
- **Classic Control:** Problemas clásicos de la literatura de RL (CartPole, MountainCar)
- **MuJoCo:** Control continuo 3D
- **Robotics:** Brazos robóticos 3D y manos robóticas
- **Toy Text:** Entornos simples con estados renderizados como texto

4.2 API de OpenAI Gym

4.2.1 Crear un entorno

env = gym.make('environment_name')

- Crea una instancia del entorno especificado
- Parámetro opcional render_mode: 'rgb_array', 'human', etc.

4.2.2 Resetear el entorno

observation, info = env.reset()

- Reinicia el entorno y devuelve la observación/estado inicial

4.2.3 Ejecutar un paso

observation, reward, terminated, truncated, info = env.step(action)

- Ejecuta la acción en el entorno
- Devuelve: nueva observación, recompensa, flags de terminación e info adicional
- done = terminated or truncated

4.2.4 Otros métodos útiles

- **env.render():** Renderiza el estado actual del entorno
- **env.close():** Cierra el entorno y libera recursos
- **env.action_space:** Información sobre el espacio de acciones
- **env.observation_space:** Información sobre el espacio de observaciones
- **env.reward_range:** Rango de recompensas posibles

4.3 Entornos personalizados

Gymnasium permite crear entornos personalizados heredando de gym.Env e implementando los métodos necesarios. Los entornos personalizados pueden registrarse y usarse con la misma API.

5. ALGORITMOS DE REINFORCEMENT LEARNING

5.1 Taxonomía de algoritmos

Los algoritmos de RL se pueden clasificar de varias formas:

5.1.1 Por tipo de método

- **Value Functions:** Buscan una función para maximizar el valor de los estados. Derivan π* de esta función.
- **Policy Search:** Buscan directamente π*. Métodos basados en gradient descent.
- **Actor-Critic:** Combinan ambos: Policy Search (actor) para aprender π* y Value Function (critic) para evaluar π.

5.1.2 Por uso del modelo

- **Model-Free:** No aprenden ni usan un modelo del entorno. Ejemplos: Q-Learning, SARSA, DQN, PPO, SAC.
- **Model-Based:** Aprenden o usan un modelo del entorno para planificar. Ejemplos: Dyna-Q, AlphaZero, World Models.

5.2 Funciones de valor

5.2.1 Función de valor de estado Vπ(s)

El valor de un estado es el retorno esperado empezando en s y siguiendo π:

$$V\pi(s) = \mathbb{E}\pi[Gt \mid St = s]$$

Recordando que Gt = Rt+1 + γGt+1, podemos expresar:

$$V\pi(s) = \Sigma a\ \pi(a|s)\ \Sigma s',r\ p(s',r|s,a)[r + \gamma V\pi(s')]$$

5.2.2 Función de valor de acción Qπ(s, a)

El valor de un par estado-acción es el retorno esperado empezando en s, tomando a, y siguiendo π después:

$$Q\pi(s, a) = \mathbb{E}\pi[Gt \mid St = s, At = a]$$

$$Q\pi(s, a) = \Sigma s',r\ p(s',r|s,a)[r + \gamma V\pi(s')]$$

Donde Vπ(s') = Σa' π(a'|s') Qπ(s', a')

5.2.3 Funciones de valor óptimas

Las ecuaciones de optimalidad de Bellman definen los valores óptimos:

$$V^*(s) = maxa\ \Sigma s',r\ p(s',r|s,a)[r + \gamma V^*(s')]$$

$$Q^*(s, a) = \Sigma s',r\ p(s',r|s,a)[r + \gamma\ maxa'\ Q^*(s', a')]$$

5.2.4 Problema: Desconocimiento del entorno

La función p (dinámica del entorno) es generalmente desconocida o muy compleja. ¿Cómo obtenemos v* y q* sin conocer p?

Solución: Estimar a partir de la experiencia.

5.3 Aprendizaje de funciones de valor desde experiencia

La regla de actualización general es:

$$nuevo_estimado \leftarrow viejo_estimado + paso \times (objetivo - viejo_estimado)$$

Para funciones de valor:

$$V_{t+1}(s) = V_t(s) + \alpha_t(G_t - V_t(s))$$

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha_t(G_t - Q_t(s, a))$$

Donde α_t es el tamaño del paso y (G_t - estimado) es el error.

5.3.1 Computando política desde funciones de valor

$$\pi(s) = \text{argmax}_a\, Q(s, a)$$

Ejemplo: Si $Q(s, \cdot) = [2, 6, -1, 4]$ para acciones [0, 1, 2, 3], entonces argmax = 1 (acción con valor máximo 6).

5.4 Exploración vs Explotación

Hay un dilema fundamental en RL:

- **Exploración:** Necesitamos explorar el espacio de soluciones (acciones aleatorias)
- **Explotación:** Necesitamos explotar el conocimiento actual (acciones greedy según la política aprendida)

5.4.1 ε-Greedy

Una técnica simple para balancear exploración y explotación:

- **Con probabilidad (1-ε):** Seguir la política $\pi(s) = \text{argmax}_a\, Q(s, a)$
- **Con probabilidad ε:** Tomar una acción aleatoria

6. MÉTODOS DE OPTIMIZACIÓN DE VALOR

6.1 Monte Carlo

Características principales:

- No requiere conocimiento previo de la dinámica del entorno
- Solo para tareas episódicas
- Basado en promediar Gt
- Los valores y políticas se actualizan al final de cada episodio completo

6.1.1 Algoritmo Monte Carlo

Para cada episodio:

9. Elegir estado inicial aleatorio S y acción inicial A
10. Generar episodio: S_0, A_0, R_1, S_1, A_1, R_2, ... ST-1, ST
11. Inicializar G = 0
12. Para t desde (longitud episodio - 1) hasta 0:
    - G = R + γG (acumular retorno hacia atrás)
    - visits[S, A] = visits[S, A] + 1
    - Q[S, A] = Q[S, A] + (1/visits[S, A]) × (G - Q[S, A])

El término 1/visits actúa como αt (paso adaptativo).

6.2 Temporal Difference (TD) Methods

Combinan características de Monte Carlo y Programación Dinámica:

- Actualizan estimados basándose en otros estimados aprendidos (bootstrap)
- No necesitan episodios completos para actualizar

6.2.1 SARSA (On-Policy TD)

Algoritmo:

13. Inicializar Q[Sterminal, ·] = 0
14. Para cada episodio:
    - S = estado inicial aleatorio
    - A = elegir acción ε-greedy desde S
    - Mientras no done:

S', R, done = ejecutar A en el entorno

A' = elegir acción ε-greedy desde S'

$$target = R + \gamma Q[S', A']$$

$$Q[S, A] = Q[S, A] + \alpha(target - Q[S, A])$$

S = S', A = A'

Nombre SARSA: Viene de la tupla (S, A, R, S', A') usada en cada actualización.

6.2.2 Q-Learning (Off-Policy TD)

Algoritmo:

15. Inicializar Q[Sterminal, ·] = 0

16. Para cada episodio:
    - S = estado inicial aleatorio
    - Mientras no done:

A = elegir acción ε-greedy desde S

S', R, done = ejecutar A en el entorno

$$target = R + \gamma \max_a Q[S', a]$$

$$Q[S, A] = Q[S, A] + \alpha(target - Q[S, A])$$

S = S'

Diferencia clave con SARSA: Q-Learning usa max sobre todas las acciones (off-policy), mientras SARSA usa la acción que realmente tomará (on-policy).

6.3 On-Policy vs Off-Policy

Característica	On-Policy	Off-Policy
Uso de datos	Usa datos de la política actual	Puede usar datos de otras políticas
Exploración	Debe balancear cuidadosamente	Puede usar política más exploratoria
Eficiencia de datos	Menor (datos no reutilizables)	Mayor (experience replay)
Sesgo/Varianza	Menor sesgo, mayor varianza	Mayor sesgo, menor varianza
Estabilidad	Más estable	Puede ser menos estable
Ejemplos	SARSA, PPO, A2C	Q-Learning, DQN, SAC

6.4 Experience Replay

Técnica para mejorar la eficiencia del uso de experiencias pasadas:

- Almacena experiencias (S, A, R, S') en un buffer
- Muestrea mini-batches aleatorios para actualizar Q
- Rompe la correlación temporal entre datos
- Proporciona muestras i.i.d. para aproximadores de función

7. APROXIMACIÓN DE FUNCIONES Y DEEP Q-NETWORK

7.1 Necesidad de aproximación de funciones

Cuando tenemos espacios de estados grandes o acciones/observaciones continuas, no podemos almacenar todos los valores en una tabla. En su lugar, usamos una forma funcional parametrizada:

$$\hat{V}(s, w) \text{ o } \hat{Q}(s, a, w)$$

donde $w \in \mathbb{R}^d$ es un vector de pesos.

7.1.1 Aproximación como aprendizaje supervisado

Es una instancia de aprendizaje supervisado con algunos problemas:

- **No estacionariedad:** El target cambia mientras aprendemos
- **Bootstrapping:** Usamos estimados para actualizar estimados

7.2 Redes Neuronales Profundas

7.2.1 El Perceptrón

Unidad básica de una red neuronal:

$$y = activation(\Sigma\, w_i x_i + b)$$

Donde x son las entradas, w los pesos, b el sesgo, y activation es una función no lineal.

7.2.2 Multi Layer Perceptron (MLP)

Múltiples capas de perceptrones conectadas:

$$y = f(x, w) = f\theta(x)$$

Las capas incluyen: capa de entrada, capas ocultas (hidden layers), y capa de salida.

7.3 Función de pérdida para RL

Para funciones de valor:

$$\mathcal{L}(wt) = \mathbb{E}[(V\pi(St) - \hat{V}(St, wt))^2] \quad (MSE)$$

$$\mathcal{L}(wt) = \mathbb{E}[(Q\pi(St, At) - \hat{Q}(St, At, wt))^2]$$

7.3.1 Actualización de parámetros

$$wt+1 = wt + \alpha \nabla wt\, \mathcal{L}(wt)$$

Que se expande a:

$$wt+1 = wt + \alpha(Ut - \hat{V}(St, wt))\nabla wt\, \hat{V}(St, wt)$$

Donde Ut es el target (Gt, Rt+1 + γGt+1, etc.).

7.4 Deep Q-Network (DQN)

DQN usa una red neuronal profunda para aproximar Q(s, a):

La red toma el estado s como entrada y produce valores Q para todas las acciones.

7.4.1 Función de pérdida de DQN

$$\mathcal{L}(wt) = \mathbb{E}[(Rt+1 + \gamma \max_a \hat{Q}(St+1, a, wt^{-}) - \hat{Q}(St, At, wt))^2]$$

Donde wt^{-} son los pesos de la red target.

7.4.2 Trucos para estabilidad

Target Network: Se usan dos redes:

- Online network: Se actualiza en cada paso
- Target network: Se actualiza periódicamente copiando los pesos de la online network

Experience Replay:

- Entrenar una NN requiere que los datos sean independientes e idénticamente distribuidos (i.i.d.)
- En RL hay correlación temporal entre datos
- Usar datos secuenciales es inestable
- Experience replay rompe esta correlación muestreando aleatoriamente del buffer

8. MÉTODOS DE POLICY GRADIENT

8.1 Policy Search

En lugar de aprender funciones de valor, optimizamos directamente una política parametrizada $\pi(a|s, \theta)$.

Objetivo: Maximizar el rendimiento, es decir, el retorno esperado.

8.1.1 Función de pérdida

$$\mathcal{L}(\theta t) = \mathbb{E}\pi[R_1 + \gamma R_2 + \gamma^2 R_3 + ...] = \text{Expected Discounted Reward}$$

8.1.2 Actualización de parámetros

$$\theta t{+}1 = \theta t + \alpha \nabla \theta t\, \mathcal{L}(\theta t) \quad \text{(Gradient Ascent)}$$

8.2 REINFORCE

Algoritmo fundamental de policy gradient.

8.2.1 Derivación del gradiente

El gradiente de la función objetivo es:

$$\nabla \theta t\, \mathcal{L}(\theta t) = \mathbb{E}\pi[\nabla \theta t\, \Sigma a\, Q\pi(St, a)\pi(a|St, \theta t)]$$

Usando el truco del log-derivative:

$$\nabla \theta t\, \mathcal{L}(\theta t) = \mathbb{E}\pi[Gt\, \nabla \theta t \log \pi(At|St, \theta t)]$$

Donde usamos que $\mathbb{E}\pi[Gt|St, At] = Q\pi(St, At)$ y que $\nabla f(x)/f(x) = \nabla \log f(x)$.

8.2.2 Regla de actualización de REINFORCE

$$\theta t{+}1 = \theta t + \alpha Gt\, \nabla \theta t \log \pi(At|St, \theta t)$$

Interpretación: Aumentamos la probabilidad de acciones que llevaron a buenos retornos (Gt alto).

8.2.3 REINFORCE con DNN

La política se representa con una red neuronal donde:

- Entrada: Estado s
- Salida: $\pi(a|s, \theta)$ - distribución de probabilidad sobre acciones
- **Capa de salida:** Necesita activación softmax porque la salida es una distribución de probabilidad

8.3 REINFORCE con Baseline

Los algoritmos de RL tienen un trade-off entre:

- **Sample Efficiency:** Capacidad de aprender con pocas interacciones
- **Stability:** Consistencia y fiabilidad del proceso de aprendizaje

8.3.1 El problema de la varianza

REINFORCE tiene alta varianza debido a que usa el retorno completo Gt.

Solución: Usar una baseline que reduce la varianza sin cambiar el valor esperado:

$$\nabla\theta t\, \mathcal{L}(\theta t) = \mathbb{E}\pi[(Gt - b(St))\, \nabla\theta t \log \pi(At|St, \theta t)]$$

La baseline puede ser cualquier función que no dependa de la acción.

9. MÉTODOS ACTOR-CRITIC

9.1 Fundamentos de Actor-Critic

Un caso especial de baseline es $b(St) = V\pi(St)$.

Como Gt es un estimado de $Q\pi(St, At)$, tenemos:

$$Gt - b(St) = Q\pi(St, At) - V\pi(St) = A\pi(St, At)$$

Advantage (Aπ): Mide cuánto mejor es una acción específica comparada con el promedio de acciones desde ese estado.

9.1.1 Funciones de pérdida

Actor Loss:

$$\mathcal{L}(\theta t) = \mathbb{E}\pi[A\pi(St, At) \log \pi(At|St, \theta t)]$$

Critic Loss:

$$\mathcal{L}(wt) = \mathbb{E}\pi[(V\pi(St) - \hat{V}(St, wt))^2]$$

9.1.2 Arquitectura

Actor-Critic tiene dos componentes:

- **Actor (Policy Network):** Aprende la política $\pi(a|s, \theta)$. Recibe gradientes $\Delta\theta$.
- **Critic (Value Network):** Aprende la función de valor $V(s, w)$. Recibe gradientes Δw.

Pueden compartir capas ocultas (hidden layers) para extraer características comunes del estado.

9.2 A2C (Advantage Actor-Critic)

Implementación síncrona de Actor-Critic usando advantage estimation.

La ventaja se estima usando TD error:

$$\delta t = Rt+1 + \gamma V(St+1) - V(St)$$

9.3 DDPG (Deep Deterministic Policy Gradient)

Diseñado específicamente para espacios de acción continuos.

9.3.1 Características principales

- Usa una política determinista $\pi\theta$ (no estocástica)
- El actor devuelve un valor único, no una distribución de probabilidad
- El actor busca directamente el máximo Q predicho por el critic:

$$\mathcal{L}(\theta t) = -\mathbb{E}\pi[Q\theta,t(st, \pi\theta,t(st))]$$

9.3.2 Exploración en DDPG

Como la política es determinista, necesita mecanismos externos de exploración:

- Ruido Ornstein-Uhlenbeck (proceso estocástico correlacionado)
- Ruido Gaussiano añadido a las acciones

DDPG es off-policy y usa experience replay para mejor eficiencia de muestras.

9.4 SAC (Soft Actor-Critic)

Algoritmo off-policy y model-free para espacios de acción continuos que incorpora maximización de entropía.

9.4.1 Componentes de SAC

- **Dos Critic Networks (Q-functions):** Para abordar el sesgo de sobreestimación. Se usa el mínimo de ambas.
- **Actor Network (Policy):** Produce acciones continuas como μ y σ.
- **Entropy Regularization:** Controlada por el parámetro de temperatura α para balancear exploración/explotación.
- **Experience Replay Buffer:** Almacena transiciones para muestrear mini-batches.

9.4.2 Funcionamiento

La función de pérdida del actor hace un trade-off entre:

- Maximizar la recompensa esperada
- Maximizar la entropía de la política (diversidad de acciones)

10. PROXIMAL POLICY OPTIMIZATION (PPO)

10.1 Motivación

PPO es un algoritmo on-policy que busca un balance entre estabilidad y eficiencia de muestras.

10.1.1 Problemas de métodos anteriores

DQN (2014):

- Inestable: cambia la red en un solo punto de datos
- Pero eficiente en datos: usa información de cada observación
- Método offline con Value Bootstrapping

TRPO (2015):

Optimiza una función objetivo sujeta a una restricción en el tamaño de la actualización:

$$max_\theta\ \mathbb{E}t[\pi\theta(at|st)/\pi\theta old(at|st) \cdot At] \quad subject\ to\ \mathbb{E}\pi[KL(\pi\theta old(\cdot|st), \pi\theta(\cdot|st))] \leq \delta$$

Difícil de implementar pero evita pasos demasiado grandes.

10.2 PPO: Clipped Objective

Definimos el ratio de política:

$$rt(\theta) = \pi\theta(at|st) / \pi\theta old(at|st)$$

10.2.1 Funciones de pérdida de PPO

Actor Loss (Clipped):

$$\mathcal{L}CLIP(\theta t) = \mathbb{E}t[min(rt(\theta)\hat{A}t, clip(rt(\theta), 1\text{-}\varepsilon, 1+\varepsilon)\hat{A}t)]$$

Donde típicamente ε = 0.2 (rango 0.8 a 1.2).

Critic Loss:

$$\mathcal{L}(wt) = \mathbb{E}\pi[(V\pi(St) - \hat{V}(St, wt))^2]$$

10.2.2 Intuición del clipping

PPO evita pasos demasiado grandes que podrían generar inestabilidad:

- Limita el cambio de política entre [1-ε, 1+ε] por actualización
- Evita ser demasiado optimista si la ventaja es positiva
- Evita ser demasiado negativo si la ventaja es negativa
- Permite reutilizar observaciones y reentrenar varias veces

10.3 Generalized Advantage Estimation (GAE)

$$At(st, at) = GAE(t) = \Sigma k{=}0{\rightarrow}\infty\ (\gamma\lambda)^k\ \delta t{+}k$$

Donde δt = rt + γV(st+1) - V(st)

GAE representa la Estimación de Ventaja Generalizada:

- λ controla el trade-off entre sesgo y varianza
- λ = 0: Solo ventajas de un paso (más sesgo)

- $\lambda = 1$: Ventajas Monte Carlo completas (más varianza)
- γ es el factor de descuento

10.4 Arquitectura de PPO

PPO usa arquitectura actor-critic donde:

- Actor: Produce μ y σ para la política estocástica
- Critic: Produce $V(s)$
- Se usa Trajectory Memory para almacenar experiencias
- Se muestrean mini-batches de la memoria

11. COMPARACIÓN DE ALGORITMOS

Algoritmo	Tipo	On/Off Policy	Advantage	Exploración	Acciones
DQN	Value-based	Off-policy	N/A	ε-greedy	Discretas
REINFORCE	Policy-based	On-policy	Monte Carlo	Estocástica	Ambas
A2C	Actor-Critic	On-policy	TD Error	Estocástica	Ambas
PPO	Policy-based	On-policy	GAE	Trust Region	Ambas
DDPG	Actor-Critic	Off-policy	TD Error	Ruido	Continuas
SAC	Actor-Critic	Off-policy	TD + Entropy	Entropía	Continuas

11.1 Ventajas y limitaciones clave

DQN

- Ventaja: Uso eficiente de experiencias pasadas mediante replay
- Limitación: Restringido a espacios de acción discretos

REINFORCE

- Ventaja: Simple de entender e implementar
- Limitación: Uso ineficiente de muestras y alta varianza

A2C

- Ventaja: Balance entre métodos de política y valor, reduce varianza
- Limitación: Requiere diseño cuidadoso de la función de ventaja

PPO

- Ventaja: Mejor eficiencia de muestras y robustez sobre policy gradients previos
- Limitación: Sensible a hiperparámetros como el ratio de clipping

12. MODEL-BASED REINFORCEMENT LEARNING

12.1 Fundamentos

En Model-Based RL, el agente aprende un 'modelo' de la dinámica del entorno que predice el siguiente estado y recompensa dados el estado actual y una acción.

12.1.1 Componentes clave

- **Model Learning:** El agente aprende las funciones de transición y recompensa del entorno.
- **Planning:** El agente utiliza el modelo aprendido para simular y evaluar acciones futuras.

12.2 Model-Based vs Model-Free

Model-Free RL

- Aprende política o función de valor directamente de la experiencia
- No tiene representación explícita de la dinámica del entorno
- Ejemplos: Q-Learning, DQN, SAC, DDPG, PPO
- Requiere muchas interacciones con el mundo real

Model-Based RL

- Primero aprende* un modelo del entorno
- Usa el modelo para planificar y simular resultados
- Más eficiente en muestras (puede generar experiencias simuladas)

*En algunos casos el modelo está preprogramado (ej: reglas de un juego).

12.3 Ventajas y desventajas

Ventajas

- **Sample Efficiency:** Aprende con menos interacciones reales usando experiencias simuladas.
- **Planning:** Capacidad de planificar con anticipación.
- **Adaptability:** Puede adaptarse rápidamente a cambios si el modelo se actualiza.

Desventajas

- **Model Accuracy:** El rendimiento depende de la precisión del modelo. Errores se propagan.
- **Complexity:** Más complejo de implementar.
- **Computational Cost:** Mayor costo computacional.

12.4 AlphaGo Zero

Modelo entrenado para jugar Go por Google DeepMind.

12.4.1 Características

- No se entrena con partidas humanas, sino jugando contra sí mismo
- Conoce las reglas del Go → model-based

- Genera árboles de movimientos basados en Q:
  - Explotación: usa el valor Q aprendido
  - Exploración: también explora ramas menos seleccionadas

12.4.2 Monte Carlo Tree Search (MCTS)

AlphaGo Zero usa MCTS con la red neuronal profunda:

- La red f produce estimaciones de política y valor
- MCTS construye un árbol de búsqueda para una política más precisa
- Diferentes versiones de f juegan entre sí
- Se selecciona la mejor para generar datos de entrenamiento

12.5 Dyna-Q

Algoritmo clásico que combina aprendizaje directo con planificación.

12.5.1 Funcionamiento

Después de cada interacción real, el agente realiza varios pasos de 'planificación':

17. Inicializar Q(s, a) y model(s, a)
18. Loop:
    - Elegir acción a con ε-greedy
    - Observar s', r
    - (Direct RL) Actualizar Q con la experiencia real
    - (Model Learning) Guardar transición: model(s, a) ← r, s'
    - (Planning) Repetir n veces: seleccionar (s, a) del modelo y actualizar Q

12.6 World Models

Aprenden una representación compacta del entorno:

- **VAE (Variational Autoencoder):** Comprime observaciones de alta dimensión (imágenes) a un espacio latente Zt.
- **RNN:** Modela la dinámica temporal en el espacio latente.
- **Controller:** Política simple (lineal) que actúa basándose en Zt y el estado RNN ht.

Permite que los agentes 'sueñen' - aprendan en entornos generados internamente.

12.7 Imagination-Augmented Agents (I2A)

Combina enfoques model-free y model-based.

12.7.1 Arquitectura

- **Model-Free Path:** Procesa la observación actual (baseline).
- **Imagination Module:**
  - Environment Model: predice estados futuros (LSTM)
  - Imagination Core: genera trayectorias 'imaginadas'
  - Rollout Encoder: procesa las trayectorias imaginadas
- **Policy Module:** Combina ambos paths para la acción final.

El agente aprende a usar las predicciones de su modelo de forma flexible.

13. TEMAS PRÁCTICOS DE RL

13.1 El Problema de Atribución Temporal

¿Cómo asignamos crédito (o culpa) a acciones individuales por una recompensa distante?

13.1.1 Por qué es difícil

- **Delayed Rewards:** El impacto de una acción puede no observarse hasta muchos pasos después.
- **Sparse Rewards:** Recompensas infrecuentes (ej: solo al final del juego).
- **Stochasticity:** Difícil distinguir si el resultado fue por buena acción o suerte.
- **Long Trajectories:** El crédito se diluye sobre muchos pasos.

13.1.2 Impacto en el aprendizaje

- Convergencia lenta
- Comportamiento supersticioso (atribuir recompensa a acciones irrelevantes)
- Alta varianza en el aprendizaje

13.1.3 Enfoques para mitigar

Bootstrapping (TD Learning): Usa estimados actuales para actualizar estimados previos.

Return-Based (Monte Carlo): Usa retornos completos observados.

Eligibility Traces: Híbrido que decae el crédito con el tiempo.

Advantage-Based: Mide cuánto mejor es una acción vs el promedio.

Model-Based: Atribución más directa mediante planificación.

13.2 Combinación de Observaciones

En muchos escenarios, las observaciones mezclan datos de diferentes tipos:

- Drone: IMU + Cámara(s)
- Robot móvil: IMU + LIDAR + SONAR

13.2.1 Desafíos

- **Dimensionalidad:** Espacio de estados de alta dimensión.
- **Normalización:** Diferencias de escala entre sensores.
- **Correlación:** Redundancias entre tipos de observación.
- **Network Architecture:** Diseño cuidadoso con ramas separadas para modalidades antes de fusión.

13.3 Simulación y Entornos

Componentes de un entorno de simulación para RL:

- **Physical Agent Model:** Modelo del robot, cinemática/dinámica, sensores, actuadores.
- **Sim. Environment Model:** Descripción de la tarea, modelado del entorno, interacciones.
- **Physics Engine:** Simulación de múltiples cuerpos, fluidos, colisiones.
- **Graphics Engine:** Renderizado de geometría, materiales, iluminación.

13.3.1 Plataformas de simulación

- **Unity ML-Agents:** Integración temprana de RL en motor de juegos. PPO, SAC implementados en PyTorch.
- **Godot RL Agents:** Interfaz entre Godot y algoritmos Python. Wrappers para SB3, RLLib.
- **MATLAB RL Toolkit:** Integración con SimMechanics, Simulink. DDPG, DQN, SAC, PPO.
- **NVIDIA Isaac Gym:** Simulación robótica con entrenamiento acelerado por GPU.

14. TEMAS AVANZADOS

14.1 Gradient-Less RL: Algoritmos Genéticos

Los Genetic Algorithms (GAs) son algoritmos de optimización inspirados en la selección natural. No requieren información de gradientes.

14.1.1 Proceso

19. Population Initialization: Generar población de políticas candidatas
20. Fitness Evaluation: Evaluar cada política por su recompensa acumulada
21. Selection: Seleccionar los mejores agentes (proporcional a fitness)
22. Mutation: Añadir ruido a los pesos
23. Crossover: Combinar agentes para producir offspring
24. Replacement: Nueva generación

14.1.2 GAs vs Gradient-Based

GAs:

- No requieren gradientes
- Exploran mediante operaciones genéticas
- Más robustos a óptimos locales
- Fácilmente paralelizables

Gradient-Based:

- Más eficientes en muestras
- Actualizaciones más informadas

14.2 Problemas relacionados con RL

14.2.1 Imitation Learning (IL)

Aprender a realizar tareas observando y mimicando comportamiento experto.

- No requiere función de recompensa
- Aprende de demostraciones
- Puede ser más eficiente pero puede sobreajustar al experto

Métodos:

- Behavior Cloning: Mapea estados a acciones con aprendizaje supervisado
- Inverse RL: Infiere la función de recompensa del experto

14.2.2 Inverse Reinforcement Learning (IRL)

Inferir la función de recompensa dado el comportamiento observado de un agente experto.

Objetivo: Encontrar R(s, a, s') que explique el comportamiento del experto.

Desafíos:

- Ambigüedad: múltiples funciones de recompensa pueden explicar el mismo comportamiento
- Complejidad computacional
- Requiere supuestos sobre la optimalidad del experto

14.2.3 Apprenticeship Learning

Combina IL e IRL: busca igualar o superar al experto aprendiendo la función de recompensa subyacente y optimizándola.

14.3 RL y Large Language Models

14.3.1 RLHF (RL from Human Feedback)

Los LLMs se entrenan inicialmente prediciendo palabras faltantes. Pero la salida puede ser inadecuada para uso humano.

Proceso:

25. Pre-entrenamiento: Predicción de texto
26. Fine-tuning supervisado con correcciones
27. Entrenamiento con RL: humanos valoran la utilidad de las respuestas
28. La red aprende a ser un asistente útil

14.3.2 Reasoning LLMs (Chain of Thought)

Los LLMs tienen dificultades con razonamiento matemático - tienden a alucinar respuestas.

Solución: Pensar paso a paso.

- Usa la salida previa como memoria para razonar
- Los pasos simples están más probablemente en la información aprendida
- Se entrena con RL donde se otorga recompensa por cada paso correcto

La tendencia actual es eliminar humanos del loop de RL y usar modelos entrenados solo para validar respuestas.

15. LIBRERÍAS DE REINFORCEMENT LEARNING

15.1 Stable Baselines 3

Implementaciones de alta calidad de algoritmos RL incluyendo PPO, A2C, DQN, TRPO.

- APIs claras y fáciles de usar
- Buena documentación e integración con Gymnasium
- Basado en PyTorch

15.2 Otras librerías importantes

- **RLLib (Ray):** Altamente escalable, amplia gama de algoritmos.
- **Dopamine (Google):** Enfoque en research, basado en TensorFlow.
- **TRFL (DeepMind):** Operaciones de bajo nivel de TF para RL.
- **Tianshou:** PyTorch, de Tsinghua University.
- **Spinning Up (OpenAI):** Excelente para aprender, con documentación educativa.
- **Acme (DeepMind):** Basado en TensorFlow, activamente mantenido.
- **CleanRL:** Implementaciones limpias de un solo archivo.

16. RESUMEN Y FÓRMULAS CLAVE

16.1 Fórmulas fundamentales

Retorno y descuento

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = R_{t+1} + \gamma G_{t+1}$$

Funciones de valor

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Ecuaciones de Bellman

$$V^*(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V^*(s')]$$

$$Q^*(s, a) = \sum_{s',r} p(s',r|s,a)[r + \gamma \max_{a'} Q^*(s', a')]$$

TD Learning

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S',A') - Q(S,A)] \quad \text{(SARSA)}$$

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)] \quad \text{(Q-Learning)}$$

Policy Gradient

$$\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_\pi[G_t \nabla_\theta \log \pi(A_t|S_t, \theta)] \quad \text{(REINFORCE)}$$

$$\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_\pi[A_t \nabla_\theta \log \pi(A_t|S_t, \theta)] \quad \text{(Actor-Critic)}$$

Advantage

$$A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$$

PPO

$$r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t)$$

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t)]$$

GAE

$$GAE(t) = \sum_{k=0\to\infty} (\gamma\lambda)^k \delta_{t+k}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

16.2 Checklist de conceptos clave

- Entender el ciclo Agent-Environment-State-Action-Reward
- Conocer la propiedad de Markov y su importancia
- Diferenciar entre políticas deterministas y estocásticas
- Entender el trade-off exploración vs explotación
- Distinguir entre on-policy y off-policy
- Comprender la diferencia entre V(s) y Q(s,a)

- Saber qué es el Advantage y para qué sirve
- Entender por qué DQN usa target network y experience replay
- Conocer el objetivo del clipping en PPO
- Distinguir Model-Free de Model-Based RL